

ORVYN Pulse

Social Media Analytics SaaS

Complete Project Documentation

MongoDB • Express.js • React • Node.js

Submitted to

Sir Zaeem Tariq

Submitted by

Group member	Student ID
Syed Mahad Ur Rayyan	b22110006158
Muhammad Uzair Madda	b22110006122
Muhammad Sami Uddin	b22110006113
Anas Ahmed Siddiqui	b22110006020

Abstract and document guide

ORVYN Pulse is a MERN application that brings social content metrics, sampled comment sentiment, historical posting patterns, and saved reports into separate brand workspaces. Its intended users are small marketing teams and agencies that need a common browser interface while keeping each client's information separate. The application provides independent registration, server sessions, owner and team permissions, a Meta connection adapter, and MongoDB persistence.

ORVYN Pulse follows a multi-tenant service model in which users manage separate brand workspaces through a shared application. The system combines centralized data storage, controlled access, and server-side processing. Instagram and Facebook accounts are connected through Meta authorization, subject to the account permissions and application configuration required by the provider.

The analytics component uses descriptive statistics and rule-based text analysis. Engagement rates account for missing reach values, while publishing-time recommendations use median historical performance and a minimum number of observations. Comment sentiment is classified using an English lexicon with simple negation handling.

Reading guide

Chapters 1 to 4 present the problem, objectives, requirements, and use cases. Chapters 5 to 11 describe architecture, data modeling, APIs, security, and analytics. Chapters 12 to 16 cover installation, configuration, operation, testing, and maintenance. The remaining chapters discuss limitations, design decisions, requirements traceability, and references.

Contents

1 Project introduction and objectives 4

2 SaaS model and project scope 5

3 Requirements and permissions..... 6

4 Use cases and user workflows 7

5 System architecture 8

6 Database design and relationships 9

7 Data dictionary 10

8 Backend API reference 11

9 Authentication and application security 13

10 Analytics methods and worked examples..... 14

11 Meta connection and synchronization..... 16

12 Windows installation and Atlas setup..... 17

13 Configuration and deployment 18

14 User manual 19

15 Testing and evaluation 20

16 Troubleshooting and maintenance 22

17 Limitations and future development 23

18 Engineering design decisions 24

19 Requirements traceability 25

20 References..... 26

1 Project introduction and objectives

Project background

Brands publish content across Instagram and Facebook but often compile their performance reports manually. A manager may copy engagement counts into a spreadsheet, compare posts informally, and summarize selected comments without recording which period or sample was used. Agencies also need to ensure that one client's data is not exposed to another client's team.

ORVYN Pulse addresses these tasks through workspace-scoped data collection and consistent calculations. Users can select a data source, platform, and reporting period, inspect content and comment summaries, and preserve the resulting analysis. ORVYN provides a practical agency context, while the implementation is an independent Internet Application Programming project.

Problem statement

The project addresses the design and implementation of a browser-based service for managing brand workspaces, collecting social content data, calculating performance indicators, and controlling access to account information. The system uses MongoDB, Express.js, React, and Node.js.

Objectives

- Implement independent accounts with secure registration and revocable sessions.
- Isolate each workspace's accounts, posts, comments, members, and reports.
- Provide owner, analyst, and viewer permissions enforced by the backend.
- Support account discovery, selection, encrypted token storage, and bounded synchronization through a Meta adapter.
- Explain engagement, sentiment, timing, and missing metric coverage without overstating their meaning.
- Provide installation guidance and automated verification of core behavior.

Expected outcome

The resulting application allows users to manage brand workspaces, authorize social accounts, review content performance, examine comment sentiment, and preserve reports. Team permissions separate administrative work from analysis and read-only access.

2 SaaS model and project scope

Why this is Software as a Service

A deployed instance hosts the React interface, Express API, analytics logic, and shared database centrally. Customers access the software in a browser. They create or join brand workspaces rather than installing an independent application for every brand. Updates and server-side synchronization can be managed at the hosted instance.

The tenant boundary is a workspace. Records are associated with workspaceId, and membership middleware authorizes the current user before access. One person may belong to several workspaces, but that does not grant access to unrelated workspaces. This shared-database design supports application-level multi-tenancy.

Included and excluded scope

Area	Delivered scope
Identity	Registration, login, current user, logout, expiring sessions
Brands and teams	Workspaces, member management, owner/analyst/viewer roles
Data ingestion	Meta authorization, account selection, and bounded imports
Analysis	Engagement, reach coverage, format comparisons, sentiment, timing
Reporting	Stored snapshots with JSON download
Operations	Local Node setup, Docker Compose, process-based sync

Subscriptions, payment collection, automated invoices, password recovery, email invitations, multi-factor authentication, account deletion workflows, and public report links are outside the current release. The agency plan is a schema option, not an implemented purchase flow.

Value and feasibility

The project brings frontend interaction, backend routing, authentication, database design, and external API integration into a single system. Separating these concerns into modules simplifies maintenance and allows the analytics component to be tested independently from provider requests.

3 Requirements and permissions

ID	Functional requirement
FR01	Register, sign in, identify the current user, and sign out.
FR02	List authorized workspaces and create an owned workspace.
FR03	Add registered teammates and remove eligible members.
FR04	Apply the selected data source consistently in analytics queries.
FR05	Authorize Meta, discover candidates, and confirm accounts.
FR06	Encrypt provider tokens and import posts and comments.
FR07	Filter analytics by source, platform, and date range.
FR08	Calculate engagement with explicit reach coverage.
FR09	Summarize sentiment and qualified posting windows.
FR10	Save and retrieve report snapshots.
FR11	Synchronize live accounts and show errors or warnings.
FR12	Disconnect accounts and remove their working data.

Permission matrix

Action	Owner	Analyst	Viewer
View analytics and reports	Yes	Yes	Yes
Save reports and request sync	Yes	Yes	No
Connect or disconnect accounts	Yes	No	No
Manage workspace members	Yes	No	No

Nonfunctional requirements

Security requires authenticated requests, server-side role checks, encrypted tokens, validated input, and controlled origins. Usability requires a responsive layout and clear empty, loading, and error states. Reliability uses unique indexes and upserts. Maintainability separates analytics, security, persistence, provider integration, and presentation. Performance is bounded by import and query limits; no throughput or availability target has been measured.

4 Use cases and user workflows

Use case UC01 Register and start a workspace

Actor: a new user. Precondition: the API can reach MongoDB. The user supplies a name, email, and password. The backend validates input, rejects duplicate email addresses, hashes the password, creates an owned workspace, and issues a session cookie. Successful registration returns the public user identity. A database failure may require retry or administrator investigation; registration is not implemented as a multi-document transaction.

Use case UC02 Analyze account content

Actor: an authorized workspace member. Preconditions: account content is available in the workspace. The member selects Live, a platform, and a reporting period. The backend retrieves authorized records and calculates engagement, content rankings, sentiment, and publishing-time summaries. If the period contains no matching records, the interface shows an empty result rather than a recommendation.

Use case UC03 Connect a social account

Actor: owner. Preconditions: Meta configuration and eligible accounts. The owner starts authorization, grants access at Meta, returns to the authenticated callback, and selects candidates. The backend validates expiring state and ownership, encrypts tokens, and starts an import. Denial, expired state, missing permissions, and unsupported metrics produce a cancellation or error path.

Use case UC04 Collaborate on reports

An owner adds an already registered teammate as an analyst or viewer. The analyst chooses filters and saves a titled report. Authorized members retrieve the stored snapshot. A viewer can read but cannot save a report or synchronize an account. Report creation captures the selected data at that time; later imports do not recalculate the saved record.

Use case UC05 Disconnect an account

The owner confirms removal. The service deletes the account and its imported posts and comments. Previously saved reports remain because they are independent snapshots. The user interface explains this retention behavior. Deleting account working data is not equivalent to deleting every historical copy or revoking provider permissions at Meta.

5 System architecture

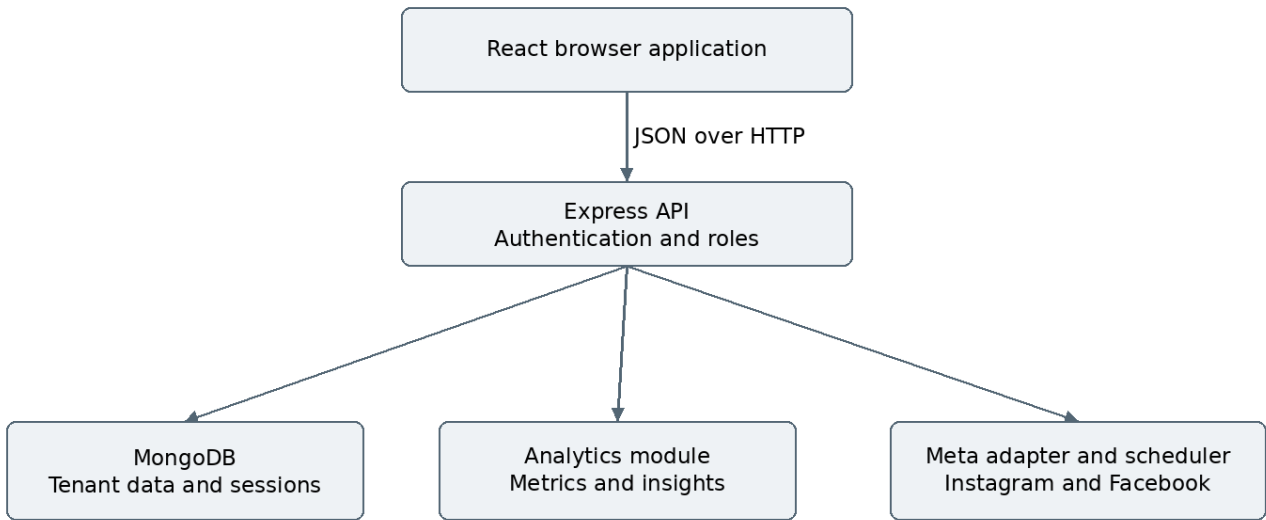


Figure 1. Main application components. Meta requests originate from Node, and imported results are persisted in MongoDB. Account tokens are not returned to the browser.

Runtime behavior

In development, Vite serves React at localhost:5173 and proxies /api to Express at port 4000. Node connects to MongoDB before starting the HTTP listener. This explains why Vite can run while API requests fail when the database is unavailable. In the build-and-start configuration, Express also serves client/dist, giving the frontend and API the same origin.

Module responsibilities

Module	Responsibility
client/src/App.jsx	Authentication views, navigation, dashboard state, dialogs
client/src/style.css	Responsive layout, typography, controls, visual styling
server/src/app.js	Routes, validation, authentication, workspace authorization
server/src/models.js	MongoDB schemas and indexes
server/src/analytics.js	Pure analytics and sentiment calculations
server/src/meta.js	Graph requests, OAuth support, pagination, sync worker
server/src/security.js	Password hashing, token generation, encryption
server/src/index.js	Configuration, database startup, HTTP server, scheduler

6 Database design and relationships

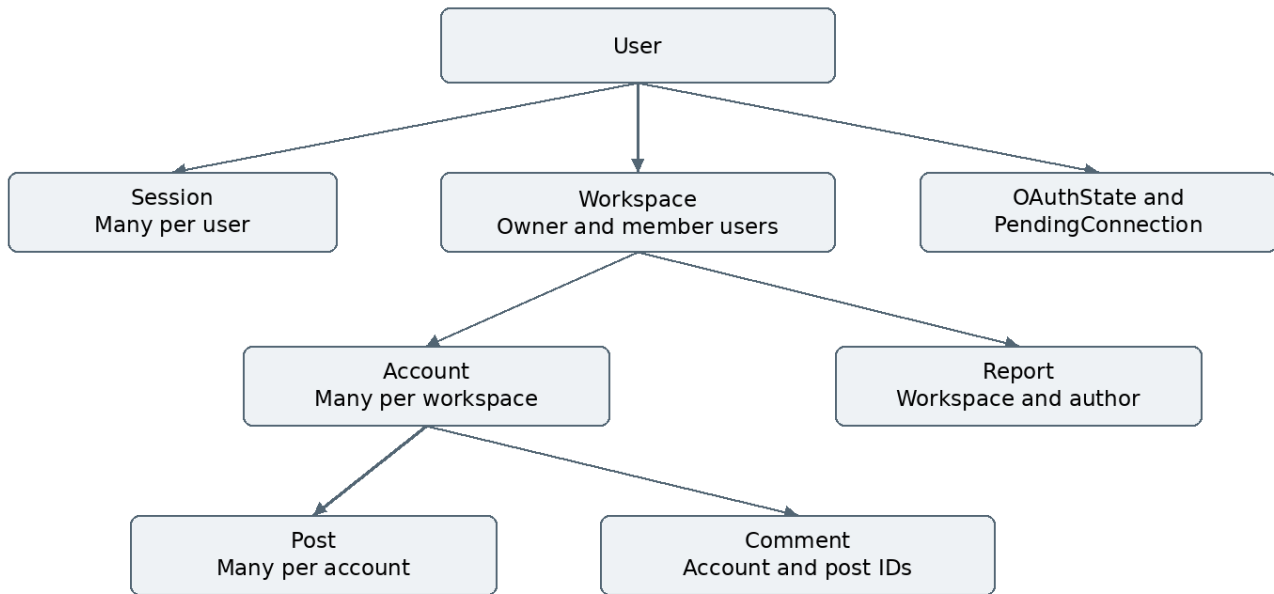


Figure 2. Logical ownership and reference structure. Members are embedded in Workspace. Comment.postId also links each comment to a post.

Modeling decisions

MongoDB stores separate collections for frequently growing records such as posts and comments. Workspace membership is embedded because the application reads roles alongside workspace access. ObjectId values link records, but MongoDB does not automatically enforce relational foreign-key constraints. The application must enforce ownership and perform cleanup.

Indexes and lifecycle

Unique indexes cover user email, session token hash, OAuth state hash, workspace/platform/remote account, account/remote post, and account/remote comment. Workspace and publication-date indexes support tenant and period queries. Session, OAuthState, and PendingConnection have expiry indexes. Authorization also checks expiry explicitly because TTL cleanup is asynchronous.

Account deletion explicitly removes posts and comments, while reports remain. Import upserts reduce duplication. Multi-step registration, account selection, and deletion are not wrapped in database transactions, so production hardening should address partial failures and concurrent operations.

7 Data dictionary

Field names below follow server/src/models.js. Date fields store timestamps; ObjectId values identify records. Mongoose timestamps add createdAt and updatedAt where enabled.

Collection	Fields and purpose
User	name; unique email; passwordHash excluded from normal query selection; timestamps.
Session	tokenHash; userId; expiresAt. The raw cookie token is not stored.
Workspace	name; timezone default Asia/Karachi; owner; members containing userId and role; plan default student; timestamps.
Account	workspaceId; platform; remotId; name; username; source; encrypted token; status; lastSync; lastError; warnings; followers; syncLockUntil; syncLock; timestamps.
Post	workspaceId; accountId; remotId; platform; source; caption; format; permalink; publishedAt; likes; commentsCount; shares; saves; reach default null; metricsUpdatedAt; timestamps.
Comment	workspaceId; accountId; postId; remotId; text; publishedAt; source; timestamps.
OAuthState	stateHash; userId; workspaceId; expiresAt. Consumed on callback.
PendingConnection	userId; workspaceId; encrypted candidates excluded from normal selection; expiresAt.
Report	workspaceId; createdBy; title; filters; data; timestamps. filters and data are flexible JSON-like objects.

Allowed values and validation

Platforms are instagram or facebook. Membership roles are owner, analyst, or viewer. Account records retain source metadata used by analytics filters. Plans are student or agency at schema level; subscription upgrades are outside the current implementation. Route validation checks identifiers, field lengths, reporting period, timezone, and accepted values.

Data retention

Working content persists until an owner disconnects its account. No automatic age-based retention policy is implemented for posts, comments, or reports. A report may include sampled comment text; report retention therefore matters when deleting personal or client information. Database backups and account-wide deletion require operational procedures beyond this version.

8 Backend API reference

All paths start with /api. Workspace routes below use W = /workspaces/:workspaceId. Authenticated routes use the pulse_session cookie. Mutations require application/json, including requests with an otherwise empty body.

Method and path	Access	Behavior
GET /health	Public	Database readiness flag and service name
POST /auth/register	Public	Create user, workspace, session; 201
POST /auth/login	Public	Validate credentials and set session
POST /auth/logout	Signed in	Revoke current session
GET /auth/me	Signed in	Return public user identity
GET /workspaces	Signed in	List accessible workspaces
POST /workspaces	Signed in	Create owned workspace; 201
GET W/accounts	Member	List workspace account metadata
DELETE W/accounts/:accountId	Owner	Delete account and working content
POST W/accounts/:accountId/sync	Owner or analyst	Start live synchronization; 202
GET W/analytics	Member	Calculate current filtered analytics
GET W/reports	Member	Latest 50 report metadata records
POST W/reports	Owner or analyst	Save calculated snapshot; 201
GET W/reports/:reportId	Member	Retrieve full report JSON
GET W/members	Member	List member names, emails, roles
POST W/members	Owner	Add registered email and role
DELETE W/members/:userId	Owner	Remove non-owner membership

Request examples

POST /api/auth/register

```
{ "name": "Project User", "email": "student@example.com",
  "password": "REPLACE_WITH_A_STRONG_PASSWORD" }
```

GET /api/workspaces/WORKSPACE_ID/analytics

?source=live&platform=all&days=30

The analytics URL above is wrapped for readability; send it as one continuous URL. Registration requires name length 2–80, email up to 180 characters, and password length 10–128.

8 Backend API reference continued

Method and path	Access	Behavior
GET W/meta/status	Member	Show configuration availability and API version
POST W/meta/connect	Owner	Create state and return authorization URL
GET /meta/callback	Signed in owner	Validate state, exchange code, discover accounts, redirect
GET W/meta/pending/:pendingId	Owner	Return token-free candidate metadata
POST W/meta/pending/:pendingId	Owner	Confirm selected accounts and start initial imports

Input contracts

Workspace creation accepts name and an IANA timezone. Member creation accepts email and role, where role is analyst or viewer. Report creation accepts a title of 1–100 characters and uses source, platform, and days from the query string. Account selection accepts an accounts array containing platform:remoteld strings that must match the server-held candidates.

For connected-account analytics, send source=live, platform=all|instagram|facebook, and an integer days value from 7 to 90. The interface offers 7, 30, and 90 days. Calculations return summary, daily, formats, times, insights, posts, comments, sentiment, sentimentTrend, filters, and generatedAt.

Error contract

Status	Meaning
400	Invalid fields, IDs, timezone, expired state, or unsupported action
401	Missing/expired session or incorrect login
403	Insufficient role or rejected origin
404	Requested authorized resource or pending selection not found
409	Duplicate email, existing membership, or unique record conflict
415	Mutation did not use JSON
429	Request rate limit exceeded
500 or 503	Unexpected backend failure or unavailable Meta configuration

Errors normally use an error text property. The API suppresses internal details for unexpected failures. A 202 sync response acknowledges starting background work; it does not prove that synchronization finished successfully. Check the account's status, last synchronization time, errors, and warnings afterward.

9 Authentication and application security

Password and session security

Passwords use Node's `scrypt` with a random 16-byte salt and a 64-byte derived key. Verification uses a timing-safe comparison. Login generates a random 32-byte token; the cookie contains that token while MongoDB stores its SHA-256 hash. Sessions expire after seven days and logout deletes the active server session.

Cookies are `HttpOnly` and `SameSite=Lax`. Secure is enabled in production, so public production deployment requires HTTPS. API responses use `Cache-Control: no-store`. General API traffic is limited to 300 requests per minute and registration/login to 30 requests per 15 minutes in the current configuration.

Authorization and browser boundaries

Every workspace request checks authenticated membership. Separate middleware checks owner or editing permissions. An attacker changing a workspace ID must still pass authorization; hiding controls in React is not the security boundary. State-changing requests require JSON and reject a supplied Origin that differs from `APP_URL`. These controls support same-origin use but do not constitute a formal security audit.

Provider secret protection

Meta tokens and pending candidates use AES-256-GCM with a 32-byte key, random 12-byte initialization vector, and authentication tag. The key is supplied through `TOKEN_ENCRYPTION_KEY`. Password hashes, account tokens, and encrypted candidates are excluded from ordinary query selection. The browser receives account metadata rather than tokens.

OAuth protection

A random state value is stored as a hash with user, workspace, and ten-minute expiry. Callback consumes it for the same signed-in user and checks continued workspace ownership. Pending account candidates expire after ten minutes. Confirmation validates selected IDs against the encrypted server-side candidates.

Operational responsibilities

Keep `.env` out of source control and submitted screenshots. Use separate development and production secrets, restrict Atlas access, and back up database data and encryption keys securely. Changing the token encryption key without migrating encrypted values prevents existing tokens from being decrypted. MFA, password recovery, audit logs, and comprehensive deletion flows remain future work.

10 Analytics methods and worked examples

Interactions and reach

For each post, interactions = likes or reactions + comment count + shares + saves. Facebook reactions occupy the likes field, and Facebook saves are not imported. Missing non-reach counters may become zero, so totals depend on the actions exposed by the platform and permissions.

Eligible posts have finite, positive reach. Reach sum adds their reach values. A person reached by two posts can be counted twice, so this is not unique account-level audience reach. Weighted engagement rate = $100 \times \text{eligible-post interactions} \div \text{eligible-post reach}$. Reach coverage = $100 \times \text{eligible-post count} \div \text{selected-post count}$.

Post	Interactions	Reach	Included in rate
A	20	100	Yes
B	40	200	Yes
C	30	Unavailable	No

Worked example: all-post interactions = 90; eligible interactions = 60; reach sum = 300; engagement rate = 20%; coverage = 66.67%. Dividing 90 by 300 would incorrectly include Post C’s interactions without its reach. When no post has usable reach, overall reach and rate are null and shown as unavailable.

Historical posting windows

The algorithm converts publication times to the workspace timezone, groups eligible posts by weekday and three-hour interval, calculates each post’s engagement rate, and takes the median within the window. At least three posts are required for a recommendation. For rates of 4%, 8%, and 12%, the window median is 8%. The highest qualified median becomes a suggested window to test.

This is an observational comparison. It does not control for content quality, paid promotion, post age, or audience changes. It does not measure when followers are online and does not guarantee future performance.

Trend and format interpretation

The content chart groups current cumulative metrics by publication date, not the date of each interaction. Synchronization can therefore change older chart points. Format recommendations compare average interactions for formats with at least three posts; the displayed average is rounded. These comparisons can favor older content.

10 Analytics methods continued

Sentiment algorithm

Comment text is lowercased and tokenized with an English-letter expression. Recognized positive words add one and negative words subtract one. A negation token in the preceding three tokens reverses a matched word's polarity. A positive total is positive sentiment, a negative total is negative sentiment, and zero is neutral.

Example comment	Score	Classification
Amazing helpful work	2	Positive
Bad and broken	-2	Negative
What is the price	0	Neutral
Not good	-1	Negative

Neutral can mean that no word was recognized. The algorithm does not reliably interpret Urdu, Roman Urdu, sarcasm, emojis, slang, or complex negation. These results are keyword-based estimates for human review, not a validated prediction model. Classification accuracy and F1 score require a labeled evaluation dataset, which is not included in the current evaluation.

Sentiment trends and samples

Each fetched comment receives a label, and counts are grouped by the comment's own publication date. Negative proportion equals negative sampled comments divided by all sampled comments. The interface must be interpreted in terms of the fetched sample, not every comment or all followers. Imported comments are capped per post and may omit substantial parts of a conversation.

Automatic insight rules

- Recommend a timing window only when a qualified bucket exists; otherwise request a stronger sample.
- Identify the leading content format by average interactions when it has at least three posts.
- Report the negative proportion when sampled comments exist.
- Explain unavailable reach when selected posts have no usable reach.

Snapshot consistency

Saving a report stores the calculated response, filters, timezone, generation time, author, and workspace. No report-update API is provided. Immutability is an application behavior, not a cryptographically signed or database-enforced guarantee. Reports may contain comment text and must remain restricted to authorized members.

11 Meta connection and synchronization

1. The signed-in owner requests a Meta authorization URL. The backend creates expiring user-bound state.
2. The owner authorizes the app on Meta. The browser returns to `/api/meta/callback` with state and code.
3. The backend consumes state, rechecks ownership, exchanges tokens, and discovers Facebook Pages and linked Instagram professional accounts.
4. The backend stores encrypted candidates temporarily. The browser sees candidate metadata only.
5. The owner selects eligible accounts. The backend validates selections, encrypts tokens, upserts account records, and starts synchronization.
6. The importer retrieves recent posts, optional metrics, and sampled comments, then records `lastSync`, warnings, and status.

Integration choice and configuration

The implemented path uses Facebook Login and Page-linked Instagram professional accounts. It does not implement the separate Instagram Login path or personal Instagram analytics. The supplied API baseline is v25.0 and is configurable. Supported versions, metric names, permissions, and review requirements must be verified for the registered Meta app before live use.

Default requested scopes are `pages_show_list`, `pages_read_engagement`, `read_insights`, `instagram_basic`, `instagram_manage_insights`, and `instagram_manage_comments`. Some Page content may require additional approved permissions. Configure the exact registered callback, app ID, secret, scopes, and encryption key in `server/.env`. Development access may be limited to authorized app testers.

Synchronization boundaries

Requests use bearer authorization, an `appsecret_proof` where configured, and a 20-second timeout.

Pagination reuses trusted Graph endpoints and cursor values instead of following arbitrary next URLs. Imports are capped at 60 posts and 50 comments per post. Optional metric failures produce warnings; missing reach remains null.

The scheduler checks due accounts every minute and uses a configurable interval, default 60 minutes with a minimum of five. A three-minute lease, renewed during processing, reduces duplicate work. Manual sync runs in the Node process. There is no durable external queue, and stopping Node stops scheduled work. Expired or revoked provider access can require reconnection.

12 Windows installation and Atlas setup

Prerequisites

Use Node.js 22.12 or newer, PowerShell, the extracted source package, and either MongoDB Atlas or a local MongoDB server. Internet access is needed to install dependencies and reach Atlas or Meta. MongoDB Compass is an optional database viewer; it is not the database server.

Install the application

```
cd "C:\path\to\orvyn-pulse"
node --version
npm.cmd --version
npm.cmd ci
node scripts/setup.mjs
```

Run commands from the directory containing the root package.json. npm.cmd avoids PowerShell script-policy issues. The setup script creates server/.env with a generated encryption key and does not overwrite an existing configuration.

Configure MongoDB Atlas

1. Open the intended Atlas project. Resume a paused cluster and wait for it to become available.
2. Open Connect and select Drivers, then Node.js. Add your current IP address to the project access list.
3. Create or use a database user with read/write access for the project database. Its password is separate from the Atlas website login.
4. Copy the Atlas connection string into MONGODB_URI in server/.env. Replace placeholders and add the database name orvyn_pulse before the question mark.
5. Save the file and restart Node. Percent-encode reserved characters in the username or password; for example, @ becomes %40.

```
MONGODB_URI=mongodb+srv://USER:PASSWORD@HOST/
  orvyn_pulse?retryWrites=true&w=majority
APP_URL=http://localhost:5173
```

The URI is wrapped above for printing. In .env it must be one continuous line, with the exact hostname and additional options supplied by Atlas. USER, PASSWORD, and HOST are placeholders, not working credentials. Never include real passwords in a submitted report.

Launch

```
npm.cmd run dev
```

Keep the terminal open and visit <http://localhost:5173>. Register an account and open Connected accounts to configure the Meta connection. Select Live when reviewing imported content. After changing server/.env, stop the process with Ctrl+C and restart it. Atlas connection guidance is listed in Chapter 20.

13 Configuration and deployment

Variable	Purpose and default
PORT	Express port; default 4000
MONGODB_URI	Atlas URI or local mongodb://127.0.0.1:27017/orvyn_pulse
APP_URL	Exact frontend origin; development http://localhost:5173
NODE_ENV	development locally; production for HTTPS deployment
TOKEN_ENCRYPTION_KEY	Unique 64 hexadecimal characters; generated by setup
META_APP_ID and META_APP_SECRET	Your registered Meta application credentials
META_API_VERSION	Configurable Graph version; supplied baseline v25.0
META_REDIRECT_URI	Registered callback; local port 4000 /api/meta/callback
META_SCOPES	Comma-separated approved permissions
SYNC_INTERVAL_MINUTES	Default 60; minimum effective interval five minutes
META_FB_REACH_METRIC	Configurable optional Facebook reach metric

Alternative local database

Install MongoDB Community Server with the Windows service option, then verify Get-Service MongoDB in PowerShell. If stopped, start it from an administrator terminal using Start-Service MongoDB. Use the default local URI above. A Compass connection to 127.0.0.1:27017 works only when a local database server is listening.

Docker installation

```
node scripts/setup.mjs
docker compose up --build
```

Docker Desktop must be running. Compose starts MongoDB and the Node application, persists database data in a named volume, and exposes the app at http://localhost:4000. The provided configuration binds to localhost and is a local HTTP configuration.

Build and production preparation

```
npm.cmd run build
npm.cmd start
```

For this local built version set APP_URL=http://localhost:4000. For public hosting use HTTPS, NODE_ENV=production, the actual APP_URL and callback, persistent MongoDB, securely managed secrets, process supervision, and backups. Review proxy trust and rate limiting for the selected host. Public deployment and operational verification are separate deployment activities.

14 User manual

Screen	User operation and expected result
Registration and login	Create an individual account or sign in. Registration password must have at least ten characters.
Workspace selector	Choose an accessible brand workspace; create another workspace if needed.
Overview	Read KPI cards, publication-date chart, leading posts, and automatic insights.
Filters	Select Live, the required platform, and 7/30/90 days. Check timezone and coverage before interpreting results.
Content performance	Inspect ranked posts, formats, interactions, and reach-based rates. Use available content filtering.
Audience sentiment	Review sample label counts, sentiment trends, and individual comment classifications.
Posting times	Read weekday and three-hour buckets. Inspect sample count and median rate.
Reports	Save a titled snapshot as owner/analyst; download authorized reports as JSON.
Connected accounts	Owner starts the Meta connection and selects eligible accounts. Review sync errors and warnings.
Team	Owner adds a registered user by email as analyst or viewer, or removes eligible members.

Recommended first session

Register and select the automatically created workspace. Connect an eligible account, wait for synchronization, and select Live. Choose the required reporting period and platform. Review reach coverage before comparing engagement rates, and check the number of observations behind timing recommendations. Save a report when the required filters are selected.

Working with teammates

Each teammate first creates an account. The owner then adds that registered email in Team. No invitation email is sent. To verify roles, use separate browser profiles or private windows. A viewer should be able to inspect data and reports but should not be able to perform owner or analyst actions.

15 Testing and evaluation

Recorded automated verification

The frontend production build completed successfully. The automated unit and HTTP suites recorded 13 passing tests: eight covering analytics and security, and five covering HTTP behavior.

Test area	Verified assertion
Reach eligibility	Missing reach excluded from numerator and denominator
No reach	Null rate and an explicit coverage insight
Timing	Three-sample rule and timezone date boundary
Sentiment	Positive, negative, neutral, and simple negation cases
Comment aggregation	Counts agree with the sentiment trend
Passwords	Salted hashes and correct-password verification
Encryption	Plaintext hidden and tampering detected
Pagination	Untrusted next URL is not fetched
HTTP access	Unauthenticated workspace request returns 401
HTTP validation	Invalid registration rejected before persistence
HTTP content type	Non-JSON mutation rejected with 415
HTTP origin	Cross-origin mutation rejected with 403
Frontend serving	Express serves built HTML with status 200

Reproduce the tests

```
npm.cmd run build
```

```
npm.cmd run test:unit
```

```
npm.cmd run test:integration
```

Build before unit tests because the HTTP frontend-serving test expects client/dist. Integration tests use mongodb-memory-server or TEST_MONGODB_URI. The suite creates and drops its own uniquely named pulse_test database; use a database user permitted to perform those test operations.

Unverified boundaries

Database integration testing was blocked by a MongoDB filesystem restriction in the test environment. The suite covers authentication, workspace isolation, role enforcement, data filtering, report persistence, token protection, OAuth state, account removal, and logout. Live Meta testing, browser end-to-end testing, load testing, and backup recovery validation remain pending.

15 Manual acceptance test plan

The following acceptance cases define the operations and expected outcomes for system-level evaluation. Execution records should include the test date, tester, observed result, and relevant log or screenshot reference. These cases are pending execution.

Case	Procedure	Expected result
AT01	Register two different users in separate browser sessions.	Each gets an independent account and owned workspace.
AT02	As user B, request a resource belonging to A without membership.	No protected records returned.
AT03	Synchronize the same connected account twice.	Stable identifiers prevent duplicate imported records.
AT04	Select Live in a workspace without connected accounts.	The result contains no imported account content.
AT05	Add a registered teammate as viewer and attempt report creation.	Read works; write is rejected by backend.
AT06	As analyst, save report; change filters and retrieve it.	Stored values and original filters remain unchanged.
AT07	Inspect timing with fewer than three eligible posts per window.	No unsupported best-window recommendation.
AT08	Log out, then retry a protected API request.	Session no longer authorizes the request.
AT09	Owner disconnects an account with a saved report.	Working account content removed; report retained.
AT10	Configure Meta and complete a real authorization with a test account.	Eligible candidates appear and selected account imports data.
AT11	Deny authorization or use expired state.	Connection fails clearly without creating an unauthorized account.
AT12	Try narrow browser width and keyboard navigation.	Core controls remain readable and usable.

16 Troubleshooting and maintenance

Symptom	Likely cause and action
npm.ps1 execution blocked	Use npm.cmd in PowerShell; no execution-policy change is needed.
package.json not found	Change into the extracted orvyn-pulse root folder.
MongoDB service not found	Local server may be missing or installed without a service. Use Community Server or configure Atlas.
Compass refuses 127.0.0.1:27017	No local server is accepting connections. For Atlas, use its cloud URI instead.
Vite runs but /api fails	The backend may not have started. Read the Node error and verify database connectivity.
Atlas authentication failed	Replace password placeholders; use database credentials; encode reserved URI characters.
Atlas connection timeout	Confirm cluster readiness, current IP access, hostname, and network reachability.
Changes to .env have no effect	Save server/.env, stop the running process, and restart npm.cmd run dev.
Origin is not allowed	APP_URL must match the browser origin exactly, including port and hostname.
Login fails on local HTTP in production mode	Secure production cookies require HTTPS. Use development settings for the local HTTP environment.
Meta Connect unavailable	Check app ID, secret, valid encryption key, and restart.
Empty analytics	Check workspace, source, platform, date range, and whether data was loaded.

Routine maintenance

Keep package-lock.json for reproducible installs. Preserve MongoDB backups and a protected copy of the encryption key. Review account synchronization errors and permission changes. Test dependency and API-version upgrades before replacing a working release. Stop the application cleanly when changing infrastructure. No automated backup or recovery service is included.

Incident investigation

Start with the backend terminal rather than repeatedly refreshing the frontend. Capture the error text without secrets, identify whether it is database, authentication, origin, provider, or client-related, and change one relevant setting at a time. Record the fix and repeat the failed action to verify recovery.

17 Limitations and future development

Current limitations

- Live Meta OAuth and synchronization require verification against approved accounts and the selected API version.
- Historical import is bounded, and deleted remote comments may remain in local working data until account removal.
- Some missing interaction counters become zero. Cross-platform totals are therefore not perfectly comparable.
- The reach total is a sum across posts, not deduplicated audience reach.
- Timing suggestions are descriptive and do not control for post age, promotion, or audience changes.
- Sentiment uses a limited English word list and has no measured classification accuracy.
- The scheduler is process-based. It does not offer durable job retries or exactly-once processing.
- Reports have no update endpoint but are not cryptographically immutable. They retain data after account removal.
- Multi-document writes are not transactional, and authorization, concurrency, and load need broader evaluation.
- Query caps can truncate large workspaces. Results are not a complete unlimited export.

Future development priorities

Priority	Enhancement	Reason
First	Complete live and database integration tests	Establish end-to-end correctness and permission behavior
First	Data retention, deletion, and audit controls	Support responsible use of real client content
Next	Durable worker queue and retries	Recover imports across process failures
Next	Historical metric snapshots	Compare equal post ages and genuine daily change
Next	Password recovery and MFA	Improve account recovery and protection
Later	Evaluated multilingual sentiment	Support Urdu and Roman Urdu with measured quality
Later	Billing and subscription entitlements	Support a paid SaaS release after operational validation

18 Engineering design decisions

Shared database with workspace isolation

The system stores tenant records in shared MongoDB collections and identifies their workspace through `workspaceId`. This structure avoids operating a separate database for every brand. It also makes authorization essential: each request must resolve membership before reading or modifying workspace data. Database-per-tenant isolation can be considered if future customers require independent backup or residency controls.

Server sessions

Opaque session tokens allow the backend to revoke the active session by deleting its stored hash. The browser does not need to manage token verification or expiry logic. This approach requires a database lookup during authenticated requests but gives the application direct control over session validity.

Provider adapter separation

Graph requests, account discovery, pagination, and synchronization are kept in `meta.js`. Route handlers coordinate authorization and persistence, while the adapter handles provider-specific behavior. A change to a metric name or account endpoint can therefore be addressed within the integration layer rather than spread throughout the interface.

Explainable calculations

Engagement and publishing-time calculations are deterministic functions of stored records. This permits direct unit testing and makes results reproducible. Median rates reduce the influence of extreme observations in a timing window, although they do not remove confounding factors such as post age or promotion.

Report snapshots

Reports store calculated values and their filters when created. This supports later review of the same reporting period even if synchronization changes the working dataset. Storage requirements increase because snapshots can duplicate content, so retention and export policies are part of future operational planning.

Bounded processing

The application limits imports and analytics queries to keep a single-process deployment manageable. Current analytics queries retrieve at most 10,000 posts and 20,000 comments. The interface lists up to 50 recent reports. For larger workspaces, paginated access, incremental aggregation, and durable background jobs would reduce request and memory pressure.

19 Requirements traceability

Requirement	Implementation	Evidence or verification route
FR01 Identity	app.js; security.js; User and Session	Unit/HTTP tests plus integration cases
FR02 Workspaces	Workspace model; access middleware	Tenant isolation integration and AT01–02
FR03 Teams	Member routes and Team screen	Role integration and AT05
FR04 Sources	source filters in app.js	Filter integration and AT04
FR05 Meta selection	OAuthState; PendingConnection; callbacks	State integration; live AT10–11 pending
FR06 Imports	meta.js; unique indexes; encryption	Encryption/pagination tests; live sync pending
FR07 Filters	analytics route; React filter controls	Manual filter comparison
FR08 Engagement	analyze in analytics.js	Passing reach eligibility and null-rate tests
FR09 Insights	sentiment and postingBucket functions	Passing timing and sentiment tests
FR10 Reports	Report model and report routes	Snapshot integration and AT06
FR11 Sync visibility	Account status, warnings, scheduler	Live synchronization acceptance pending
FR12 Removal	Account deletion route	Cleanup integration and AT09

Course relevance

The implementation covers the principal Internet Application Programming topics: client-server communication, authentication, access control, persistent data, request validation, external APIs, and responsive user interfaces. The traceability matrix links the functional requirements to their implementation and verification routes.

Project conclusion

ORVYN Pulse provides a MERN architecture for managing brand workspaces and analyzing social content. Access control, encrypted provider credentials, consistent metric definitions, and persistent report snapshots form the core of the system. Further development focuses on integration validation, reliable background processing, broader language support, and operational controls.

20 References

Primary project evidence

The project source is organized into server, client, tests, and configuration files. Core references include `server/src/app.js`, `models.js`, `security.js`, `analytics.js`, `meta.js`, and `index.js`; `client/src/App.jsx`; `README.md`; `Dockerfile`; `compose.yaml`; and the automated test suites.

Official technical references

React documentation

<https://react.dev/>

Express documentation

<https://expressjs.com/>

Node.js API documentation

<https://nodejs.org/api/>

MongoDB database documentation

<https://www.mongodb.com/docs/>

Mongoose documentation

<https://mongoosejs.com/docs/>

MongoDB Atlas connection prerequisites

<https://www.mongodb.com/docs/atlas/connect-to-database-deployment/>

MongoDB Atlas driver connection guide

<https://www.mongodb.com/docs/atlas/driver-connection/>

Meta Facebook Login manual flow

<https://developers.facebook.com/docs/facebook-login/guides/advanced/manual-flow/>

Instagram API with Facebook Login

<https://developers.facebook.com/docs/instagram-platform/instagram-api-with-facebook-login/>

Meta Pages API

<https://developers.facebook.com/docs/pages-api/>

Meta Graph API changelog

<https://developers.facebook.com/docs/graph-api/changelog/>